ZeroController: Real-Time Body-Controlled 2D Fighting Game

Final Semester Project Report

Abdul Rahman Azam 23K-0061
Muhammad Umar 23K-0023
Talha Rusman 23K-0065

Supervisor: Dr Kamran Ali

May 9, 2026

Abstract

ZeroController is a webcam-based human action recognition system that allows a player to control a 2D fighting game using body motion instead of a standard controller. The final system captures pose landmarks with MediaPipe, converts them into short temporal sequences, classifies the action with a lightweight Spatial-Temporal Graph Convolutional Network (ST-GCN), and sends the predicted command to the game in real time. This report summarizes the task definition, dataset, preprocessing, architecture, loss, hyperparameters, and a comparison with state-of-the-art skeleton-action recognition methods.

1 Task Definition

The project addresses **skeleton-based temporal action recognition** for **real-time human-in-the-loop game control**. The input is a live webcam stream. Each frame is converted into a 33-joint body skeleton using MediaPipe Pose, and a short sequence of consecutive skeletons is classified into one of nine game actions:

`idle, left_punch, right_punch, left_kick, right_kick, jump, block, forward, backward`

This is not object detection or semantic segmentation because the model does not predict bounding boxes or pixel masks. It is also not temporal action localization in the standard video-understanding sense, because the system works on a fixed rolling window and outputs the current action directly for online game control. Therefore, the most accurate task description is:

online skeleton-sequence classification for low-latency action-to-command mapping

2 Dataset Description

2.1 Data format

The training data is collected from a webcam using the custom data-collection tool in the project. Instead of training directly on raw RGB video, the final classifier uses **pose sequences**. Each sample is stored as a NumPy file with shape:

$$(T, V, C) = (30, 33, 4)$$

where:

- $T = 30$ is the temporal length in frames,
- $V = 33$ is the number of MediaPipe body landmarks,
- $C = 4$ contains $(x, y, z, visibility)$ for each joint.

2.2 Labels

The dataset uses **class-category labels only**. There are no bounding boxes, masks, keyframe timestamps, or tracking identities in the training labels. Each saved sequence belongs to exactly one action class.

2.3 Collected dataset summary

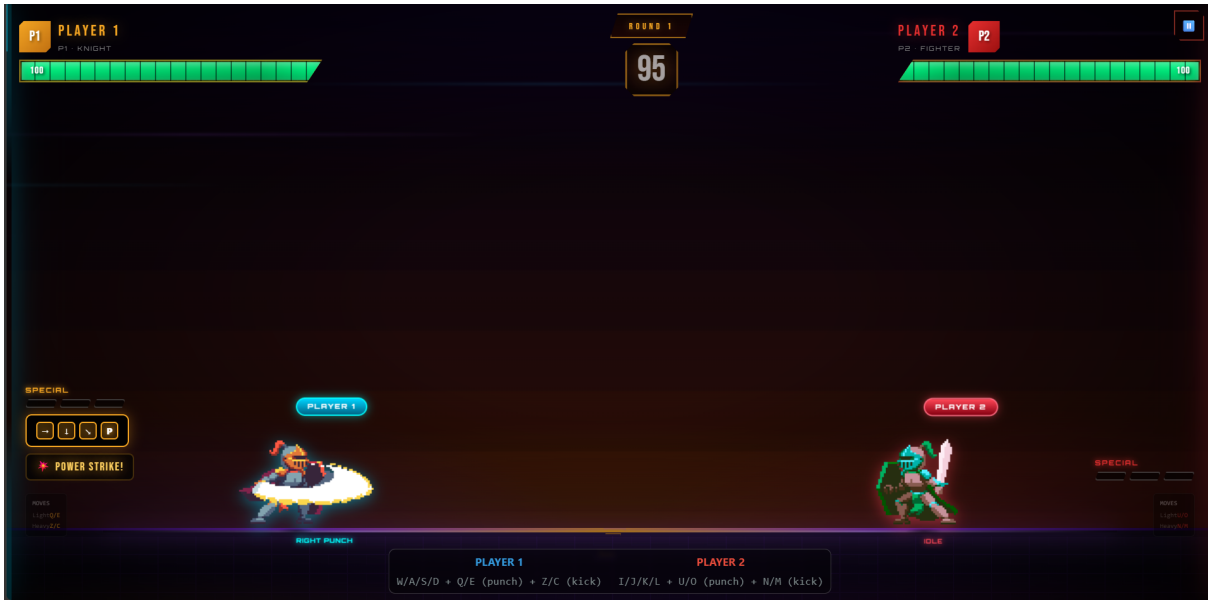
At the time of writing, the raw dataset contains **631 labeled sequences**. The current class distribution is shown in Table 1.

Table 1: Raw dataset distribution collected from webcam sessions.

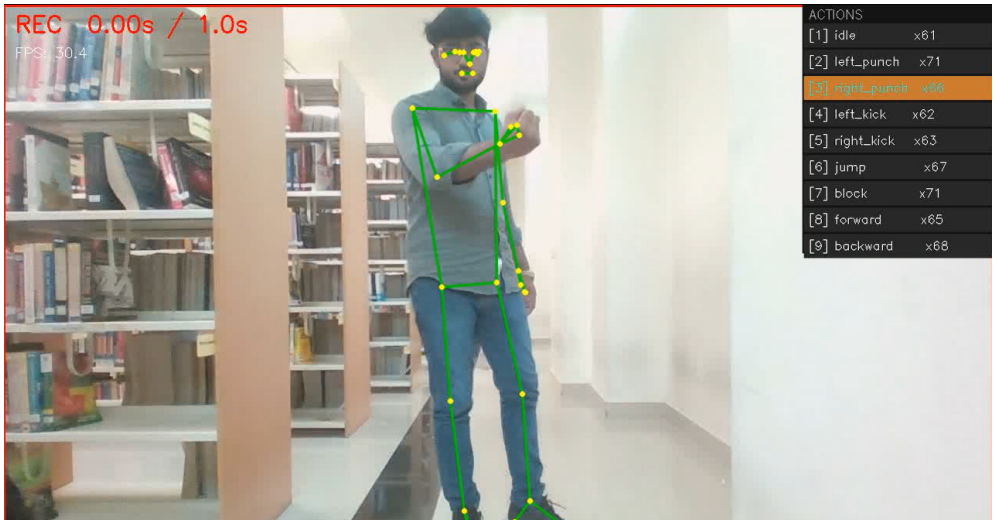
Action Class	Samples
idle	58
left_punch	71
right_punch	71
left_kick	71
right_kick	71
jump	76
block	71
forward	71
backward	71
Total	**631**

2.4 Game screenshots

Figure 1 shows the 2D game interface and the real-person pose capture used to control the game.



(a) 2D fighting game controlled by predicted body actions



(b) Real-person pose capture and action recording interface

Figure 1: Project screenshots showing the game view and the webcam-based pose-control interface.

3 Data Pre-processing

The preprocessing pipeline has two stages: conversion from webcam frames to skeletons, and normalization of skeleton sequences for the neural network.

3.1 Pose extraction

Each RGB frame is passed through the MediaPipe Pose Landmarker. Only one pose is tracked at a time. The model outputs 33 landmark points with normalized coordinates and a visibility score. These landmarks are appended to a rolling sequence buffer.

3.2 Sequence preparation

Each training sample contains 30 frames, corresponding to approximately one second of motion at 30 FPS. This is long enough to capture punches, kicks, blocking, and stepping actions while still keeping the latency acceptable for gameplay.

3.3 Normalization and feature engineering

The implemented preprocessing steps are:

1. **Hip-centering**: the midpoint of the hips is subtracted from all joints, making the representation independent of where the player stands inside the camera frame.
2. **Scale normalization**: the skeleton is normalized using torso length so the model is less sensitive to body size and camera distance.
3. **Visibility weighting**: each joint coordinate is weighted by MediaPipe visibility to reduce the impact of low-confidence or occluded joints.
4. **Velocity features**: the first temporal difference $\Delta(x, y, z)$ is appended, so the ST-GCN receives both posture and motion information.

After preprocessing, each sample becomes a tensor of shape:

$$(C, T, V) = (6, 30, 33)$$

where the six channels are the three position coordinates and the three velocity coordinates.

3.4 Data augmentation

The project also includes an augmentation script to improve generalization. The implemented augmentations are:

- left-right mirroring of pose sequences,
- spatial noise,
- skeleton scaling,
- temporal jitter,
- visibility dropout,
- small rotation jitter,
- time stretching,

- same-class mixup.

These augmentations are especially useful because the dataset is manually collected and therefore much smaller than public research benchmarks.

4 Network Architecture

The final selected architecture is a **Spatial-Temporal Graph Convolutional Network (ST-GCN)**. This choice fits the problem because the human body is naturally a graph: joints are nodes and bones are edges.

4.1 Architecture overview

Figure 2 shows the full system pipeline from webcam input to game action output.

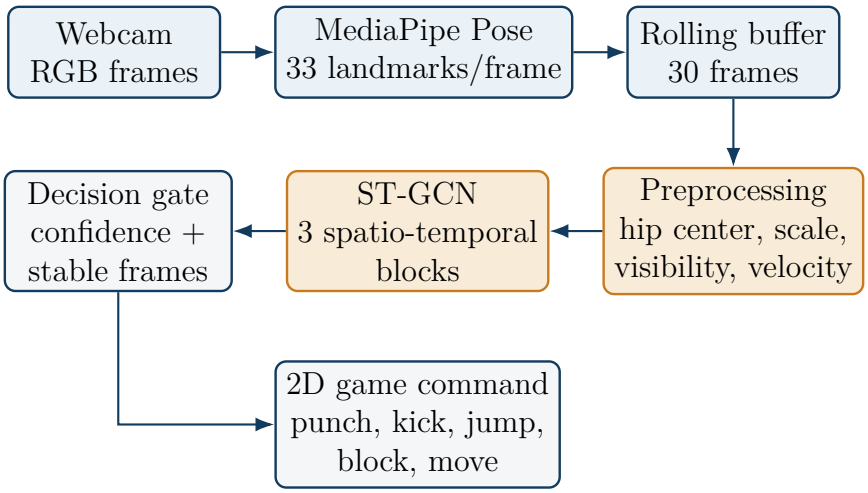


Figure 2: End-to-end ZeroController inference pipeline.

4.2 Graph construction

The skeleton graph contains 33 nodes and uses the standard human-body bone connections defined by MediaPipe Pose. A symmetric adjacency matrix is built once, self-loops are added, and the matrix is normalized using:

$$\hat{A} = D^{-1/2}(A + I)D^{-1/2}$$

This normalized adjacency matrix allows each joint to aggregate information from its neighboring joints without allowing high-degree joints to dominate the representation.

4.3 ST-GCN block design

Each ST-GCN block contains:

- a spatial graph convolution,
- batch normalization,

- ReLU activation,
- a temporal convolution,
- dropout,
- a residual or skip connection.

The current configuration uses three blocks with channel progression:

$$6 \rightarrow 32 \rightarrow 64 \rightarrow 64$$

followed by global average pooling over time and joints, and a final linear classifier producing 9 logits.

4.4 Why ST-GCN was selected

The repository also includes LSTM and TCN baselines, but ST-GCN was selected as the main model for three reasons:

1. it explicitly models skeleton topology,
2. it is lightweight enough for real-time CPU inference,
3. it is better suited to small manually collected skeleton datasets than larger transformer-style models.

5 Loss Function

The training objective is a **single-term multi-class classification loss**. Since each 30-frame sequence belongs to exactly one action class, the model is trained with categorical cross-entropy:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{9} y_{ic}\log(p_{ic})$$

where:

- N is the batch size,
- y_{ic} is the one-hot target label,
- p_{ic} is the predicted softmax probability for class c.

This project does **not** use a compound loss such as classification plus localization, because the task is pure sequence classification. Therefore:

- number of loss terms: **1**
- classification loss weight: **1.0**
- all other terms: **not used**

6 Hyperparameters

The main hyperparameters used by the current implementation are listed in Table 2.

Table 2: Training and inference hyperparameters used in the implemented system.

Hyperparameter	Value	Reason / selection method
Sequence length	30 frames	Manual selection; enough to cover a full action while remaining responsive.
Batch size	8	Chosen manually for small dataset stability and limited hardware.
Epochs	120	Manual tuning to allow convergence without excessive overfitting.
Learning rate	1×10^{-3}	Standard Adam starting point; adjusted from common deep-learning practice.
Optimizer	Adam	Reliable default for small and medium supervised classification problems.
LR scheduler	Cosine annealing, $\eta_{\min} = 10^{-5}$	Smooth decay generally improves final validation accuracy.
Dropout	0.3	Regularization against overfitting on a manually collected dataset.
Model type	ST-GCN	Selected after architectural reasoning for skeleton data.
ST-GCN channels	(32, 64, 64)	Lightweight channel growth for real-time inference.
Temporal kernel	9	Covers sufficient short-term motion context.
Use velocity stream	True	Adds explicit motion information beyond pose alone.
Confidence threshold	0.75	Tuned manually for stable gameplay without suppressing valid actions.
Stable frames	2	Reduced latency while still filtering flicker.
Trigger cooldown	170 ms	Prevents repeated accidental command firing.

Hyperparameter selection methodology: because the project uses a custom local dataset rather than a large public benchmark, the hyperparameters were chosen using a combination of literature-guided defaults, manual tuning, and practical latency constraints. In particular, the architecture follows common skeleton-action-recognition practice, while gameplay-specific thresholds were tuned empirically to make the system responsive and stable during live use.

7 SOTA Comparison

7.1 Important comparison note

A direct numerical comparison between this project and published state-of-the-art methods is **not fully apples-to-apples**, because this project is trained on a small custom webcam dataset, while published SOTA results are usually reported on large public benchmarks such as NTU RGB+D 60 and NTU RGB+D 120. Therefore, the comparison below should be interpreted as:

- a **benchmark-level literature comparison** of architectures,
- plus a **deployment-level comparison** of speed, complexity, and suitability for this project.

7.2 Quantitative benchmark comparison

Table 3 summarizes representative top-1 results reported in the literature for skeleton-based action recognition on NTU RGB+D 60.

Table 3: Representative literature comparison on NTU RGB+D 60. These numbers come from published benchmark reports and are used only for architectural context, not as a direct same-dataset comparison with ZeroController.

Method	Year	Cross-Subject	Cross-View
ST-GCN	2018	81.5%	88.3%
MS-G3D	2020	91.5%	96.2%
CTR-GCN	2021	92.4%	96.8%
InfoGCN	2022	92.7%	96.9%
Recent transformer / hybrid models	2023–2025	≈92.7–93.6%	≈96.5–96.9%

7.3 Qualitative comparison

Compared with large benchmark models, ZeroController makes a different engineering tradeoff:

- **Strength**: lower complexity and lower latency, which is critical for real-time body-controlled gameplay.
- **Strength**: easier training on a small custom dataset.
- **Limitation**: lower expected recognition ceiling than larger multi-stream or transformer-based SOTA models.
- **Limitation**: custom dataset size is much smaller than public benchmark datasets, so generalization is naturally more limited.

7.4 Project-vs-SOTA conclusion

For a final-year project with live interaction requirements, ST-GCN is a strong and technically justified compromise. It does not aim to surpass benchmark SOTA models trained on massive datasets. Instead, it prioritizes:

1. real-time inference,
2. correct use of skeleton-graph structure,
3. compatibility with manually collected training data,
4. practical game-control stability.

8 Conclusion

ZeroController demonstrates a complete deep-learning pipeline for real-time body-driven game control: webcam capture, pose extraction, sequence formation, graph-based temporal classification, and stable command dispatch to a 2D game. The technical core of the project is a lightweight ST-GCN operating on normalized 33-joint pose sequences. The design is appropriate for the task, computationally practical, and aligned with modern skeleton-based action-recognition research.

The main limitation is dataset scale: the system is trained on a custom webcam dataset that is far smaller than public skeleton-action benchmarks. Even with that constraint, the final design remains technically sound because it balances recognition quality, low latency, and deployability in a live 2D game setting.